

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1407
Design

COURSE NAME: Compiler

COURSE OUTCOME COVERED

CO4: Examine intermediate code generation techniques to enhance code optimization (BL4)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

Code of Conduct:

I Santhosh Kumar /192411015 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: V.Santhosh Kumar

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

Annexure A

INDUSTRY PROBLEM- SOLVING TASK

1. Banking Transaction Processing System

A banking software company is developing a compiler for a domain-specific scripting language used in transaction processing systems. The compiler must translate variable declarations, assignment statements, and Boolean expressions efficiently to avoid transaction failures during peak hours. The system also uses case statements to classify transactions such as deposit, withdrawal, and loan processing. During testing, incorrect intermediate code generation caused delays in transaction execution.

Tasks:

- (i) Design suitable intermediate code for the given assignment and Boolean expressions. (K3)
- (ii) Explain how backpatching can help in handling conditional transaction validation. (K4)
- (iii) Analyze the role of intermediate languages in improving banking software portability. (K5)

Answer:

(i) Intermediate Code:

```
t1 = balance - amount
if amount > balance goto L1
balance = t1
goto L2
L1: call printMessage("Insufficient Funds")
goto L3
L2: t2 = txnType
    if t2 == 'DEPOSIT' goto D1
    if t2 == 'WITHDRAW' goto D2
    if t2 == 'LOAN' goto D3
D1: balance = balance + amount ; goto L3
D2: balance = balance - amount ; goto L3
D3: call processLoan(accountID, amount) ; goto L3
L3:
```

(ii) Explanation:

Backpatching helps handle conditional transaction validation as follows:

- The Boolean condition (e.g. `amount > balance`) is translated before the target labels of the true/false branches are known, so the jump instructions are generated with an empty target field.
- The compiler maintains `truelist` and `falselist` arrays of these incomplete jump instructions for each condition.
- Once the code for the validation-success block and the rollback block is generated, `backpatch()` fills in the actual addresses into the pending jump instructions.

- This avoids a two-pass compilation and lets the compiler emit code in a single left-to-right pass, which is essential to avoid the delays seen during peak-hour transaction processing.
- It also allows nested/compound Boolean conditions (AND/OR of multiple checks) to be merged efficiently before the final patch.

(iii) Analysis:

Role of intermediate languages in improving banking software portability:

- The IR decouples the front end (parsing the scripting language) from the back end (target machine/VM code), so the same IR can be retargeted to different banking server platforms without rewriting the whole compiler.
- Machine-independent optimizations (constant folding, dead-code elimination) can be applied once on the IR and benefit every target platform.
- A stable IR makes the compiler modular, so new transaction types (e.g. UPI, wallet transfer) can be added without disturbing the code-generation back end.
- IR-level static analysis allows correctness/security checks before final code generation, which is valuable for regulated banking software.

2. Smart Traffic Control System

An automotive company is building a smart traffic management application where signals are controlled automatically based on traffic density. The compiler used in the embedded controller must process Boolean expressions and case statements for different traffic conditions. Due to inefficient intermediate code generation, the response time of traffic signals increased significantly during rush hours. The development team plans to optimize procedure calls and assignment statements in the compiler design.

Tasks:

- Construct intermediate code for the traffic signal control logic. *(K3)*
- Explain how procedure calls improve modularity in the traffic control application. *(K4)*
- Evaluate the importance of compiler optimization in real-time embedded systems. *(K5)*

Answer:

(i) Intermediate Code:

```
t1 = trafficDensity
if t1 == LOW goto L1
if t1 == MEDIUM goto L2
if t1 == HIGH goto L3
goto L4
L1: call setSignal(GREEN, 10) ; goto Lexit
L2: call setSignal(GREEN, 20) ; goto Lexit
L3: call setSignal(GREEN, 40) ; goto Lexit
L4: call setSignal(RED, 5)
Lexit:
```

(ii) Explanation:

How procedure calls improve modularity in the traffic control application:

- `setSignal(colour, duration)` is compiled once as a callable procedure and invoked with `param/call` three-address instructions from every density-based branch, instead of duplicating the signal-switching logic in each branch.
- A single change to the signal-timing algorithm (e.g. adding a pedestrian phase) only needs the procedure body to be updated, not every call site.
- Keeping the logic in procedures reduces compiled code size, which matters for the limited memory of an embedded traffic controller.
- Procedures can be unit-tested and verified in isolation before being integrated into the full control system.

(iii) Analysis:

Importance of compiler optimization in real-time embedded systems:

- Optimizations such as constant folding and common sub-expression elimination reduce the number of instructions executed in every control cycle, directly lowering signal response latency.
- Removing redundant density re-evaluations shortens the polling loop that reads the sensors.
- Smaller, optimized code fits within the limited ROM/RAM of the embedded controller.
- Predictable, tightly-optimized execution timing is essential to meet the real-time deadlines needed to avoid the increased response time seen during rush hours.

3. E-Commerce Product Recommendation Engine

An e-commerce company uses a rule-based recommendation engine developed using a custom programming language. The compiler processes declarations, assignment statements, and Boolean conditions to generate recommendations dynamically. During seasonal sales, delays occurred because of improper handling of case statements and inefficient intermediate code generation. The organization wants to redesign the compiler to improve execution speed and maintainability.

Tasks:

- (i) Generate intermediate representation for the recommendation logic. *(K3)*
- (ii) Discuss how case statements can efficiently handle product category selection. *(K4)*
- (iii) Analyze the role of compiler design in improving large-scale e-commerce applications. *(K5)*

Answer:

(i) Intermediate Code:

```

t1 = category
if t1 == 'ELECTRONICS' goto L1
if t1 == 'FASHION' goto L2
if t1 == 'GROCERY' goto L3
goto L4
L1: score = baseScore * 1.2 ; goto Lend
L2: score = baseScore * 1.1 ; goto Lend

```

```
L3: score = baseScore * 1.05 ; goto Lend
L4: score = baseScore
Lend: if score > threshold goto Lrecommend
```

(ii) Explanation:

How case statements can efficiently handle product category selection:

- The case statement is compiled into a chain of conditional jumps (or a jump table when categories map to dense integer codes), sending control directly to the scoring rule for the matched category.
- This avoids evaluating a long if-else-if ladder for every product, reducing average comparisons under heavy load such as seasonal sales.
- A jump-table implementation gives near-constant-time dispatch when there are many categories.
- New categories can be added as new case branches without modifying the logic of existing branches.

(iii) Analysis:

Role of compiler design in improving large-scale e-commerce applications:

- Efficient IR generation and optimization phases lower the CPU cost per recommendation request, letting the engine serve more concurrent users during peak sales.
- Clean handling of declarations, assignments and case statements in the compiler makes the rule engine easier to extend with new recommendation rules.
- Compile-time type/declaration checking catches errors before deployment, preventing the runtime failures that previously caused delays.
- Optimized generated code reduces the memory footprint of the rule engine when it is replicated across distributed servers.

4. Hospital Patient Monitoring System

A healthcare technology company is developing a patient monitoring system that continuously checks vital signs using embedded software. The compiler used in the system must efficiently evaluate Boolean expressions and assignment statements for emergency alerts. Improper backpatching in conditional branching caused delayed alarm generation during critical patient conditions. The software team is redesigning the compiler for faster and more reliable intermediate code execution.

Tasks:

- (i) Design intermediate code for patient condition monitoring logic. *(K3)*
- (ii) Explain the significance of backpatching in emergency alert systems. *(K4)*
- (iii) Evaluate how compiler optimization improves reliability in healthcare applications. *(K5)*

Answer:

(i) Intermediate Code:

```
t1 = heartRate < 60
t2 = heartRate > 140
t3 = t1 or t2
if t3 goto L1
```

```
goto L2
L1: alarmStatus = 1 ; call triggerAlarm(patientID) ; goto L3
L2: alarmStatus = 0
L3:
```

(ii) Explanation:

Significance of backpatching in emergency alert systems:

- The compound Boolean condition (bradycardia OR tachycardia) produces jump instructions whose target (the alarm-triggering block) is not yet known at the point they are generated.
- The truelist/falselist of these pending jumps is backpatched with the correct address once the alarm block is emitted, in a single forward pass over the source.
- This guarantees the condition-to-alarm wiring is correct on the first pass, which is critical since a mis-linked jump could delay or skip a life-critical alert.
- It keeps code generation fast, avoiding the extra translation overhead that could otherwise add to alarm latency.

(iii) Analysis:

How compiler optimization improves reliability in healthcare applications:

- Eliminating redundant or dead condition checks on the sensor-sampling hot path reduces the instructions executed on every monitoring cycle.
- Shorter, optimized code reduces the time between a vital-sign threshold breach and the alarm firing, directly improving patient safety.
- Better register allocation reduces unnecessary memory traffic on the embedded monitor, improving timing determinism.
- Optimized, well-verified code lowers the risk of the type of intermediate-code errors that previously caused delayed alarm generation.

5. Online Airline Reservation System

An airline company is modernizing its reservation platform using a custom scripting language for ticket booking and seat allocation. The compiler must handle declarations, assignment statements, and procedure calls efficiently to manage thousands of booking requests simultaneously. During testing, errors in intermediate code generation affected seat allocation and payment confirmation processes. The software architects aim to improve compiler performance for large-scale distributed applications.

Tasks:

- Develop intermediate code for ticket booking and seat allocation operations. *(K3)*
- Explain how procedure calls support modular reservation management. *(K4)*
- Analyze the role of intermediate languages in distributed airline reservation systems. *(K5)*

Answer:

(i) Intermediate Code:

```
call checkSeatAvailability(flightID, seatClass)
t1 = availableSeats
```

```

if t1 == 0 goto Lfull
param passengerID ; param flightID ; param seatClass
call bookSeat, 3
t2 = fare * taxRate
totalFare = fare + t2
goto Lend
Lfull: call notifyWaitlist(passengerID)
Lend:

```

(ii) Explanation:

How procedure calls support modular reservation management:

- Booking, availability-checking and payment are each compiled as separate procedures (checkSeatAvailability, bookSeat, processPayment) using standard param/call three-address code.
- The same procedures are reused for every booking request instead of duplicating logic per flight, reducing compiled code size.
- Because each call is a self-contained unit with its own activation record, thousands of simultaneous bookings can be handled without one request's logic interfering with another's.
- Each module (search, book, pay) can be tested and modified independently, which simplifies maintenance.

(iii) Analysis:

Role of intermediate languages in distributed airline reservation systems:

- A machine-independent IR lets the same reservation logic be compiled and deployed across the different servers/architectures of a distributed booking cluster.
- Optimizations such as inlining small procedures and eliminating redundant seat-lookup calls can be applied on the IR before final code generation, cutting latency under load.
- A stable IR supports partial recompilation when only one module (e.g. payment) changes, which is valuable in a large distributed codebase.
- IR-level analysis can check for consistency/concurrency issues before generating platform-specific code, reducing risks such as double booking.

Set 2

1. Industrial Robot Automation System

A manufacturing company is developing an industrial robot control system where robotic arms perform assembly operations automatically. The compiler used in the embedded software processes declarations, assignment statements, and Boolean expressions to coordinate robotic movement and safety checks. During production, delays occurred due to inefficient handling of procedure calls and intermediate code generation. The company plans to redesign the compiler to improve execution speed and synchronization between robotic units.

Tasks:

- (i) Design intermediate code for robotic movement and safety validation logic. (K3)
- (ii) Explain how procedure calls improve modularity in industrial automation systems. (K4)
- (iii) Analyze the role of compiler optimization in improving manufacturing efficiency. (K5)

Answer:

(i) Intermediate Code:

```
t1 = sensorDistance < safeLimit
if t1 goto Lstop
param armID ; param targetX ; param targetY
call moveArm, 3
goto Lend
Lstop: call emergencyStop(armID)
Lend:
```

(ii) Explanation:

How procedure calls improve modularity in industrial automation systems:

- moveArm() and emergencyStop() are compiled once as callable procedures and invoked with param/call instructions from every coordination point, instead of inlining the same motion/safety code repeatedly.
- This keeps the controller program compact, which helps synchronize many robotic arms operating together.
- A single fix to the movement or safety-check logic automatically applies everywhere the procedure is called.
- High-level task sequencing can call low-level motion primitives, giving a clean layered design.

(iii) Analysis:

Role of compiler optimization in improving manufacturing efficiency:

- Removing redundant safety-boundary checks and inlining small, frequently-called movement procedures cuts execution time on every control cycle.

- Faster generated code lets robotic arms react more quickly to safety-sensor input, reducing production stoppages.
 - Efficient register allocation reduces memory-access latency, improving synchronization precision between robotic units.
 - Overall assembly-line throughput improves once the compiler-level delays that previously caused production slowdowns are eliminated.
-

2. Cybersecurity Intrusion Detection System

A cybersecurity firm is developing an intrusion detection system that continuously monitors network traffic for suspicious activities. The compiler processes Boolean expressions, assignment statements, and case statements to identify different categories of cyberattacks. During real-time monitoring, inefficient intermediate code generation increased system response time and delayed threat detection. The development team intends to enhance compiler efficiency for faster and more reliable security analysis.

Tasks:

- Generate intermediate representation for network threat detection logic. (K3)
- Discuss how case statements can efficiently classify multiple cyberattack scenarios. (K4)
- Evaluate the importance of compiler optimization in real-time cybersecurity applications. (K5)

Answer:

(i) Intermediate Code:

```
t1 = packetRate > threshold
t2 = attackType
if t2 == 'DOS' goto L1
if t2 == 'PORTSCAN' goto L2
if t2 == 'MALWARE' goto L3
goto L4
L1: if t1 goto Lalert ; goto Lend
L2: severity = 2 ; goto Lend
L3: severity = 3 ; goto Lend
L4: severity = 1
Lalert: call raiseAlert(sourceIP, severity)
Lend:
```

(ii) Explanation:

How case statements can efficiently classify multiple cyberattack scenarios:

- The case statement compiles to a dispatch chain (or jump table) keyed on attackType, sending each detected pattern directly to its specific handling code.
- This avoids evaluating a long if-else-if ladder for every packet, cutting average comparisons under continuous high-volume traffic.

- New attack categories can be added as new case branches without touching existing detection logic.
- Severity/response assignment stays isolated per category, keeping the classification logic easy to audit.

(iii) Analysis:

Importance of compiler optimization in real-time cybersecurity applications:

- Optimized IR reduces the instruction count on the packet-inspection hot path, directly lowering detection latency.
 - Common sub-expression elimination on repeated Boolean checks (rate, size, signature match) avoids recomputation for every packet.
 - Efficient code generation is essential to keep pace with line-rate network traffic without dropping packets.
 - A faster detection-to-alert pipeline shortens the window an attacker has before a response is triggered, addressing the earlier increase in response time.
-

3. Digital Payment Gateway System

A fintech company is building a digital payment gateway to process online transactions securely and efficiently. The compiler used in the transaction engine must evaluate Boolean conditions, assignment statements, and procedure calls during payment verification and fraud detection. During peak transaction periods, improper backpatching caused delays in payment confirmation and rollback operations. The organization plans to redesign the compiler to improve transaction reliability and scalability.

Tasks:

- Construct intermediate code for payment verification and fraud detection logic. *(K3)*
- Explain how backpatching supports conditional transaction processing. *(K4)*
- Analyze the role of compiler design in ensuring secure and scalable fintech applications. *(K5)*

Answer:

(i) Intermediate Code:

```
t1 = amount <= balance
t2 = fraudScore < riskLimit
t3 = t1 and t2
if t3 goto Lapprove
goto Ldecline
Lapprove: call debitAccount(accountID, amount)
          status = "APPROVED" ; goto Lend
Ldecline: call logFraudAlert(accountID)
          status = "DECLINED"
Lend:
```

(ii) Explanation:

How backpatching supports conditional transaction processing:

- The AND of the two Boolean conditions (sufficient funds, acceptable fraud score) generates jump lists whose true/false targets (approve/decline blocks) are not known until those blocks are emitted later.
- Backpatching fills in these jump addresses once the approve and decline code exists, so the compiler can emit code in a single forward pass.
- Short-circuit evaluation means a failed fraud check can skip the funds check entirely, saving cycles during peak load.
- Correctly patched jumps keep the rollback path linked so declined transactions immediately branch to rollback rather than falling through to the approve block.

(iii) Analysis:

Role of compiler design in ensuring secure and scalable fintech applications:

- A clean separation between condition evaluation, procedure calls and code generation makes it easier to verify transaction logic for correctness and security.
 - Optimizations such as dead-branch elimination and inlining small verification procedures reduce per-transaction processing time, supporting scalability under peak load.
 - A modular compiler lets fraud-rule updates be compiled independently of the core payment engine.
 - IR-level analysis can catch unreachable or duplicate validation branches before deployment, reducing the risk of the delays previously caused by improper backpatching.
-

4. Smart Agriculture Monitoring System

An agritech company is developing a smart agriculture platform that monitors soil moisture, temperature, and irrigation systems automatically. The compiler in the embedded controller processes declarations, assignment statements, and Boolean expressions to activate irrigation based on environmental conditions. Inefficient handling of intermediate code caused delays in irrigation control during critical farming conditions. The software team aims to improve compiler performance for real-time agricultural automation.

Tasks:

- Develop intermediate code for irrigation control and environmental monitoring. *(K3)*
- Explain the role of Boolean expressions in automated farming systems. *(K4)*
- Evaluate how compiler optimization enhances efficiency in smart agriculture applications. *(K5)*

Answer:

(i) Intermediate Code:

```

t1 = soilMoisture < moistureThreshold
t2 = temperature > tempThreshold
t3 = t1 and (not t2)
if t3 goto Lirrigate
goto Lskip
Lirrigate: call activateValve(zoneID)
           irrigationStatus = 1 ; goto Lend
Lskip: irrigationStatus = 0
Lend:

```

(ii) Explanation:

Role of Boolean expressions in automated farming systems:

- Boolean expressions combine multiple sensor readings (moisture, temperature, rainfall) into a single condition that decides whether to irrigate a zone.
- Compiled short-circuit jump code evaluates only as many sensor checks as necessary, saving processing time on the embedded controller.
- Complex agronomic rules (e.g. don't irrigate if moisture is low but temperature/rain conditions say otherwise) can be encoded directly in the compiled condition logic.
- Correct evaluation timing prevents both over- and under-irrigation, protecting crop yield.

(iii) Analysis:

How compiler optimization enhances efficiency in smart agriculture applications:

- Optimized generated code shortens the sense-decide-actuate loop so irrigation valves respond promptly to changing field conditions.
- Eliminating redundant re-evaluation of unchanged sensor values across cycles reduces embedded CPU load and power draw, important for field-deployed, often solar-powered controllers.
- Efficient code reduces the delays that previously caused irrigation to run late during critical dry periods.
- Smaller, optimized code fits the constrained memory of low-cost agritech embedded boards.

5. Cloud-Based Video Streaming Platform

A multimedia company is developing a cloud-based video streaming platform that dynamically adjusts video quality based on network conditions. The compiler used in the streaming engine processes case statements, assignment statements, and procedure calls for resource allocation and bandwidth management. During high user traffic, inefficient intermediate code generation caused buffering delays and reduced streaming quality. The company plans to redesign the compiler to improve scalability and performance.

Tasks:

- Design intermediate code for bandwidth allocation and quality adaptation logic. (K3)

(ii) Explain how case statements help manage multiple streaming quality levels. (K4)

(iii) Analyze the significance of compiler optimization in cloud-based multimedia systems. (K5)

Answer:

(i) Intermediate Code:

```
t1 = bandwidth
if t1 >= 5000 goto L1
if t1 >= 2000 goto L2
if t1 >= 500 goto L3
goto L4
L1: quality = "1080p" ; goto Lset
L2: quality = "720p" ; goto Lset
L3: quality = "480p" ; goto Lset
L4: quality = "240p"
Lset: call setStreamQuality(userID, quality)
```

(ii) Explanation:

How case statements help manage multiple streaming quality levels:

- Bandwidth ranges compile into a chain of conditional branches (or a jump table) that select the correct quality tier directly, avoiding repeated re-checks.
- The dispatch keeps quality-adaptation logic centralized and extensible, so a new tier (e.g. 4K) is just one added case.
- Fast dispatch matters because quality must be re-evaluated frequently as network conditions fluctuate for many concurrent viewers.
- Separating bandwidth measurement from quality-selection logic keeps the code maintainable.

(iii) Analysis:

Significance of compiler optimization in cloud-based multimedia systems:

- Optimized IR lowers the CPU cost per stream, letting the platform serve more concurrent viewers on the same server capacity.
- Reducing instruction count in the resource-allocation/quality-adaptation path directly cuts the buffering caused by slow re-encoding decisions.
- Procedure-call optimization (e.g. inlining small helper calls) reduces overhead when quality changes happen very frequently under high traffic.
- Scalable, optimized compiled code is essential for a platform experiencing bursty, high-volume simultaneous demand.

Set 3

1. Autonomous Drone Delivery System

A logistics company is developing an autonomous drone delivery system to transport packages across urban areas. The compiler embedded in the drone controller processes declarations, assignment statements, and Boolean expressions to manage navigation, obstacle

detection, and battery monitoring. During testing, inefficient intermediate code generation caused delays in route adjustment and emergency landing procedures. The organization plans to optimize the compiler for reliable real-time drone operations.

Tasks:

- (i) Design intermediate code for drone navigation and obstacle detection logic. (K3)
- (ii) Explain how Boolean expressions support autonomous decision-making in drones. (K4)
- (iii) Analyze the importance of compiler optimization in real-time delivery systems. (K5)

Answer:

(i) Intermediate Code:

```
t1 = distanceToObstacle < safeDistance
t2 = batteryLevel < minBattery
if t1 goto Lavoid
if t2 goto Lland
goto Lcontinue
Lavoid: newHeading = currentHeading + turnAngle ; goto Lend
Lland: call initiateEmergencyLanding() ; goto Lend
Lcontinue: position = position + velocity * dt
Lend:
```

(ii) Explanation:

How Boolean expressions support autonomous decision-making in drones:

- Boolean conditions (obstacle proximity, battery level) compile into jump instructions that let the drone choose between continue/avoid/land entirely from the compiled on-board logic, with no external control loop needed.
- The compiled short-circuit code checks the most safety-critical condition first, ensuring the fastest possible response to an imminent obstacle.
- Combining multiple sensor Booleans lets the drone make compound decisions (e.g. avoid only if not already landing) in a single evaluated expression.
- Reliable, correctly generated Boolean code is essential since there is no human in the loop to catch mistakes.

(iii) Analysis:

Importance of compiler optimization in real-time delivery systems:

- Optimized code minimizes the decision-loop cycle time between sensor reading and control action, which is critical at drone flight speeds.
- Eliminating redundant recomputation of the obstacle-distance expression across iterations reduces onboard processor load.
- Optimized, compact code helps meet the strict real-time deadlines and limited resources of drone controllers.
- This reduces the route-adjustment and emergency-landing delays observed during testing.

2. Railway Reservation and Scheduling System

A railway management company is modernizing its reservation and scheduling platform using a custom programming language. The

compiler processes declarations, assignment statements, case statements, and procedure calls for seat allocation, train scheduling, and passenger status updates. During peak booking hours, improper intermediate code generation caused delays in reservation confirmation and ticket cancellation. The development team aims to redesign the compiler to improve scalability and transaction efficiency.

Tasks:

- (i) Construct intermediate code for reservation and scheduling operations. (K3)
- (ii) Discuss how case statements efficiently handle passenger ticket categories. (K4)
- (iii) Evaluate the role of compiler optimization in large-scale transportation systems. (K5)

Answer:

(i) Intermediate Code:

```
t1 = ticketClass
if t1 == 'AC' goto L1
if t1 == 'SLEEPER' goto L2
if t1 == 'GENERAL' goto L3
goto L4
L1: fare = baseFare * 2.5 ; goto Lset
L2: fare = baseFare * 1.5 ; goto Lset
L3: fare = baseFare ; goto Lset
L4: fare = baseFare
Lset: param passengerID ; param trainID ; param ticketClass
      call reserveSeat, 3
```

(ii) Explanation:

How case statements efficiently handle passenger ticket categories:

- The ticket class value compiles into a branch/jump-table dispatch, sending fare calculation and seat-pool lookup directly to the matching category's logic.
- This avoids long if-else chains that would otherwise slow processing during high-volume booking windows.
- New fare classes can be added as new case labels without disturbing existing category logic.
- Category-specific rules (fare, quota) stay isolated, which makes them easy to audit and update.

(iii) Analysis:

Role of compiler optimization in large-scale transportation systems:

- Optimizing the generated reservation/scheduling code reduces per-request latency, essential when thousands of bookings arrive concurrently during peak hours.
 - Eliminating redundant procedure-call overhead (e.g. inlining small validation calls) speeds up confirmation and cancellation processing.
 - Efficient intermediate code supports horizontal scaling across multiple booking servers without a proportional rise in per-request cost.
 - This addresses the delays in reservation confirmation and ticket cancellation caused by inefficient generated code.
-

3. Intelligent Energy Management System

An energy company is building a smart grid monitoring system that automatically controls electricity distribution based on power consumption. The compiler in the embedded controller evaluates Boolean expressions and assignment statements to monitor load balancing and fault detection. During high-demand periods, inefficient backpatching delayed fault recovery and power redistribution. The company plans to enhance compiler performance to improve reliability and energy efficiency.

Tasks:

- (i) Generate intermediate code for load balancing and fault detection logic. (K3)
- (ii) Explain the significance of backpatching in smart energy management systems. (K4)
- (iii) Analyze how compiler optimization improves the performance of smart grids. (K5)

Answer:

(i) Intermediate Code:

```
t1 = load > maxCapacity
t2 = faultSensor == 1
if t2 goto Lfault
if t1 goto Lredistribute
goto Lnormal
Lfault: call isolateFaultSegment(segmentID) ; goto Lend
Lredistribute: call shiftLoad(segmentID, targetSegment) ; goto Lend
Lnormal: status = "STABLE"
Lend:
```

(ii) Explanation:

Significance of backpatching in smart energy management systems:

- The fault and overload Boolean checks generate jump instructions before the fault-isolation and load-redistribution blocks exist in the code stream.
- Backpatching fills in these jump targets once those blocks are generated, letting the compiler emit code in a single forward pass.
- This ensures the fault-handling branch is wired to execute immediately, addressing the earlier issue of delayed fault recovery caused by inefficient patching.
- It also supports combined conditions (fault OR overload) without needing the target addresses to be known in advance.

(iii) Analysis:

How compiler optimization improves the performance of smart grids:

- Optimized code reduces the compute time of the load-balancing/fault-detection loop that must run continuously across grid segments.
- Removing redundant sensor-read and recompute instructions lowers the latency between a fault occurring and corrective action.
- Efficient code generation improves reliability of automatic power redistribution during high-demand periods.
- Scalable optimized IR supports monitoring more grid segments without a proportional rise in processing overhead.

4. AI-Based Fraud Detection System

A financial institution is developing an AI-based fraud detection platform that continuously analyzes transaction patterns. The compiler processes declarations, assignment statements, and Boolean conditions to identify suspicious activities and trigger alerts. During large-scale transaction processing, inefficient procedure calls increased response time and delayed fraud notifications. The software architects plan to redesign the compiler for faster analysis and improved scalability.

Tasks:

- (i) Develop intermediate code for fraud detection and alert generation logic. (K3)
- (ii) Explain how procedure calls improve modularity in fraud detection systems. (K4)
- (iii) Evaluate the role of compiler design in secure financial applications. (K5)

Answer:

(i) Intermediate Code:

```
t1 = txnAmount > avgTxnAmount * multiplier
t2 = txnLocation != lastKnownLocation
t3 = t1 or t2
if t3 goto Lsuspicious
goto Lnormal
Lsuspicious: param accountID ; param txnID
            call flagTransaction, 2
            call triggerAlert(accountID) ; goto Lend
Lnormal: status = "CLEAR"
Lend:
```

(ii) Explanation:

How procedure calls improve modularity in fraud detection systems:

- flagTransaction() and triggerAlert() are compiled as independent callable procedures invoked (via param/call) wherever suspicious activity is detected.
- This avoids duplicating alert-handling logic across every rule that can flag a transaction.
- Procedures can be updated or extended (e.g. a new alert channel) independently of the pattern-analysis logic.
- Keeping the program modular aids testing of detection rules and response actions separately.

(iii) Analysis:

Role of compiler design in secure financial applications:

- Efficient handling of procedure calls (activation records, parameter passing) lowers response latency during large-scale transaction analysis.
- Optimizations like inlining frequently-called small procedures reduce the overhead that previously delayed fraud notifications.
- Modular, well-optimized compiler design supports scaling detection to higher transaction volumes without redesigning the core engine.

- A clear IR-level structure aids auditability, which is important for regulatory compliance in financial software.

5. Smart Warehouse Inventory Management System

A retail company is implementing a smart warehouse inventory system that automatically tracks product movement and stock levels. The compiler processes assignment statements, Boolean expressions, and case statements to update inventory records and manage automated storage systems. During seasonal sales, delays in intermediate code execution affected stock synchronization and order fulfillment. The company plans to optimize the compiler to improve inventory accuracy and warehouse efficiency.

Tasks:

- (i) Design intermediate code for inventory tracking and stock update operations. (K3)
- (ii) Explain how case statements help manage multiple product categories efficiently. (K4)
- (iii) Analyze the importance of compiler optimization in warehouse automation systems. (K5)

Answer:

(i) Intermediate Code:

```
t1 = productCategory
if t1 == 'PERISHABLE' goto L1
if t1 == 'ELECTRONICS' goto L2
if t1 == 'APPAREL' goto L3
goto L4
L1: stock[binID] = stock[binID] - qty
    call checkExpiry(binID) ; goto Lend
L2: stock[binID] = stock[binID] - qty
    call checkFragileHandling(binID) ; goto Lend
L3: stock[binID] = stock[binID] - qty ; goto Lend
L4: stock[binID] = stock[binID] - qty
Lend:
```

(ii) Explanation:

How case statements help manage multiple product categories efficiently:

- Category-specific handling (expiry checks, fragile handling) is dispatched via case branches straight to the relevant routine instead of checking every rule for every item.
- Common stock-update logic stays shared while category-specific steps remain isolated per branch.
- Fast dispatch matters when thousands of SKU movements are processed during seasonal-sale spikes.
- New categories can be added without modifying existing category logic.

(iii) Analysis:

Importance of compiler optimization in warehouse automation systems:

- Optimized generated code speeds up the inventory-update path that runs on every item movement, reducing the stock-synchronization delays seen during sales peaks.

- Eliminating redundant array-index/bounds computations lowers per-transaction overhead in the automated storage system.
- Efficient code generation supports higher transaction throughput without additional hardware.
- It improves overall inventory accuracy by ensuring updates complete promptly under load.

authentication and result generation. During large-scale online examinations, inefficient intermediate code execution caused delays in question loading and score processing. The organization plans to optimize the compiler to improve scalability and performance.

Tasks:

- (i) Construct intermediate code for student authentication and evaluation logic. (K3)
- (ii) Discuss how case statements help manage different examination categories. (K4)
- (iii) Evaluate the importance of compiler optimization in large-scale online examination systems. (K5)

Answer:

(i) Intermediate Code:

```
param studentID ; param password
call authenticate, 2
t1 = authResult
if t1 == 0 goto Ldeny
t2 = examCategory
if t2 == 'MCQ' goto L1
if t2 == 'DESCRIPTIVE' goto L2
goto L3
L1: score = autoGrade(answers) ; goto Lend
L2: score = 0 ; call queueForManualGrading(studentID) ; goto Lend
L3: score = 0 ; goto Lend
Ldeny: call denyAccess(studentID)
Lend:
```

(ii) Explanation:

How case statements help manage different examination categories:

- The exam category (MCQ vs. descriptive vs. practical) is dispatched via case branches straight to the correct grading routine.
- This avoids evaluating irrelevant grading logic for every submission, which matters when many students submit concurrently.
- New exam types can be added without altering existing category-handling code.
- Auto-grading and manual-queue logic stay cleanly separated.

(iii) Analysis:

Importance of compiler optimization in large-scale online examination systems:

- Optimized authentication and evaluation code reduces per-student processing time, critical when many students log in and submit simultaneously.
- Efficient handling of frequent authentication procedure calls lowers the login/question-loading delays seen during real exams.
- Streamlined IR for scoring logic speeds up result generation right after the exam window closes.
- Scalable, optimized code supports growth in student numbers without a proportional rise in infrastructure cost.

3. Intelligent Weather Forecasting System

A meteorological department is developing an intelligent weather forecasting application that analyzes environmental data in real time. The compiler processes assignment statements and Boolean expressions to monitor temperature, humidity, and rainfall conditions. During severe weather conditions, improper backpatching delayed alert generation and emergency notifications. The software engineers aim to redesign the compiler to improve prediction accuracy and system responsiveness.

Tasks:

- (i) Generate intermediate code for weather monitoring and alert generation logic. (K3)
- (ii) Explain the role of backpatching in handling emergency weather alerts. (K4)
- (iii) Analyze how compiler optimization enhances real-time forecasting applications. (K5)

Answer:

(i) Intermediate Code:

```
t1 = windSpeed > stormThreshold
t2 = rainfall > floodThreshold
t3 = t1 or t2
if t3 goto Lalert
goto Lnormal
Lalert: alertLevel = "SEVERE"
      call sendEmergencyNotification(region) ; goto Lend
Lnormal: alertLevel = "NORMAL"
Lend:
```

(ii) Explanation:

Role of backpatching in handling emergency weather alerts:

- The OR of the storm and flood Boolean conditions produces jump lists that must eventually point to the alert-notification block, which is generated later in the code stream.
- Backpatching resolves these jump addresses once the alert block is emitted, so target locations do not need to be known in advance.
- Each sensor condition can be processed independently and their true/false lists merged efficiently before the final patch.
- Correctly patched jumps ensure the emergency notification fires immediately when severe conditions are detected, addressing the earlier delayed-alert issue.

(iii) Analysis:

How compiler optimization enhances real-time forecasting applications:

- Optimized code reduces the processing time of the continuous environmental-data evaluation loop, keeping predictions timely.
- Eliminating recomputation of unchanged Boolean sub-expressions across consecutive readings lowers CPU load.
- Faster generated code shortens the gap between a severe-condition threshold being crossed and the alert being issued.
- It supports scaling the system to monitor more regions/sensors without slowing the response time.

4. Automated Library Management System

A university is implementing an automated library management system to handle book issuance, returns, and digital catalog management. The compiler processes declarations, assignment statements, and procedure calls for member verification and book tracking operations. During semester examinations, inefficient intermediate code generation delayed transaction updates and fine calculations. The university plans to improve compiler efficiency for faster and more reliable library services.

Tasks:

- (i) Develop intermediate code for book issue and return operations. *(K3)*
- (ii) Explain how procedure calls improve modularity in library management systems. *(K4)*
- (iii) Evaluate the significance of compiler optimization in educational information systems. *(K5)*

Answer:

(i) Intermediate Code:

```
param memberID
call verifyMembership, 1
t1 = verifyResult
if t1 == 0 goto Ldeny
param bookID ; param memberID
call issueBook, 2
dueDate = currentDate + loanPeriod
goto Lend
Ldeny: call denyIssue(memberID)
Lend:
```

(ii) Explanation:

How procedure calls improve modularity in library management systems:

- verifyMembership(), issueBook(), returnBook() and calculateFine() are each compiled as independent callable procedures invoked via param/call.
- The same verification/issue routines are reused for every transaction rather than duplicated per book or member.
- Fine-calculation logic can be modified independently of the issue/return logic.
- The modular structure simplifies adding new services (e.g. digital catalog search) without disturbing existing procedures.

(iii) Analysis:

Significance of compiler optimization in educational information systems:

- Optimized code reduces per-transaction overhead in the issue/return/fine-calculation path, addressing the delays seen during semester examinations.
- Inlining or streamlining frequently called small procedures cuts activation-record overhead under high concurrent usage.
- Efficient generated code supports timely transaction updates during peak library-usage periods.
- Scalable optimized IR helps the system handle growth in students and books without a redesign.

5. Industrial Water Quality Monitoring System

An environmental engineering company is developing a water quality monitoring system that continuously checks pH levels, contamination, and chemical composition in industrial plants. The compiler in the embedded controller evaluates Boolean expressions, assignment statements, and case statements for automated monitoring and alert generation. During critical contamination events, inefficient intermediate code execution delayed warning notifications and corrective actions. The company aims to redesign the compiler to improve reliability and response time.

Tasks:

- (i) Design intermediate code for contamination detection and alert handling logic. *(K3)*
- (ii) Explain how case statements efficiently manage different water quality conditions. *(K4)*
- (iii) Analyze the importance of compiler optimization in industrial environmental monitoring systems. *(K5)*

Answer:

(i) Intermediate Code:

```
t1 = pHLevel
if t1 < 6.5 goto L1
if t1 > 8.5 goto L2
goto Lnormal
L1: status = "ACIDIC" ; call triggerAlert("LOW_PH") ; goto Lend
L2: status = "ALKALINE" ; call triggerAlert("HIGH_PH") ; goto Lend
Lnormal: status = "SAFE"
Lend:
```

(ii) Explanation:

How case statements efficiently manage different water quality conditions:

- pH/contamination ranges compile into branch conditions that dispatch directly to the matching status/alert routine.
- Each condition's alert handling stays isolated, so acidic, alkaline and contamination cases can each trigger a tailored response.
- This avoids evaluating every possible condition sequentially for each sensor reading, useful in continuous monitoring.
- New quality parameters (e.g. turbidity) can be added as additional case branches.

(iii) Analysis:

Importance of compiler optimization in industrial environmental monitoring systems:

- Optimized code shortens the sensor-evaluation-to-alert pipeline, directly addressing the delayed warning notifications seen during contamination events.
- Removing redundant threshold recomputation across polling cycles reduces embedded-controller load.
- Efficient generated code improves the reliability of continuous, unattended monitoring in industrial plants.

- It supports scaling to more monitoring points without a proportional increase in processing delay.

Set 5

1. Smart Parking Management System

A smart city development company is designing an automated parking management system to monitor vehicle entry, slot allocation, and parking fee calculation. The compiler embedded in the control software processes declarations, assignment statements, Boolean expressions, and case statements to manage parking availability and security verification. During peak hours, inefficient intermediate code generation

caused delays in slot assignment and payment confirmation. The company plans to redesign the compiler to improve real-time response and system efficiency.

Tasks:

- (i) Design intermediate code for parking slot allocation and fee calculation logic. (K3)
- (ii) Explain how Boolean expressions and case statements help manage parking operations efficiently. (K4)
- (iii) Analyze the role of compiler optimization in improving smart parking management systems. (K5)

Answer:

(i) Intermediate Code:

```
t1 = slotStatus
if t1 == 'FREE' goto L1
goto Ldeny
L1: t2 = paymentVerified
    if t2 goto Lassign
    goto Ldeny
Lassign: slotStatus = 'OCCUPIED'
        fee = ratePerHour * duration ; goto Lend
Ldeny: call denyEntry(vehicleID)
Lend:
```

(ii) Explanation:

How Boolean expressions and case statements help manage parking operations efficiently:

- The case statement dispatches on slot status (free/reserved/occupied) to decide eligibility, and a Boolean check then verifies payment before assignment.
- Combining case dispatch with a Boolean guard avoids assigning a slot before payment or security verification completes.
- Fast, direct branching is important at entry gates where vehicles must be processed quickly during peak hours.
- New slot states or verification rules can be added without restructuring existing logic.

(iii) Analysis:

Role of compiler optimization in improving smart parking management systems:

- Optimized code speeds up the slot-allocation/payment-confirmation path, addressing the delays observed during peak-hour entry.
- Reducing redundant status re-checks across consecutive vehicle entries lowers per-vehicle processing time.
- Efficient generated code supports scaling to more parking slots/gates without slowing response.
- It improves the real-time responsiveness of security verification integrated with slot assignment.

2. Digital Healthcare Appointment System

A healthcare organization is developing a digital appointment management platform to schedule doctor consultations and manage

patient records. The compiler processes declarations, assignment statements, and procedure calls to handle appointment booking, cancellation, and patient verification. During high patient traffic, inefficient intermediate code execution caused delays in appointment confirmation and report generation. The development team plans to optimize the compiler for faster and more reliable healthcare services.

Tasks:

- (i) Construct intermediate code for appointment scheduling and patient verification logic. (K3)
- (ii) Explain how procedure calls improve modularity in healthcare management systems. (K4)
- (iii) Evaluate the importance of compiler optimization in digital healthcare applications. (K5)

Answer:

(i) Intermediate Code:

```
param patientID
call verifyPatient, 1
t1 = verifyResult
if t1 == 0 goto Ldeny
param doctorID ; param slotTime
call checkAvailability, 2
t2 = slotFree
if t2 == 0 goto Lunavailable
param patientID ; param doctorID ; param slotTime
call bookAppointment, 3
goto Lend
Ldeny: call denyBooking(patientID) ; goto Lend
Lunavailable: call suggestAlternateSlot(doctorID)
Lend:
```

(ii) Explanation:

How procedure calls improve modularity in healthcare management systems:

- verifyPatient(), checkAvailability() and bookAppointment() are each independent callable procedures reused for every booking request.
- This avoids duplicating verification/scheduling logic per doctor or department.
- Procedures can be updated (e.g. adding a new verification step) without touching the booking/cancellation logic.
- It supports a clean separation of patient-record handling from scheduling logic, aiding maintainability.

(iii) Analysis:

Importance of compiler optimization in digital healthcare applications:

- Optimized code reduces per-request latency in the scheduling/verification pipeline, addressing the delays seen during high patient traffic.
 - Efficient parameter-passing and activation-record handling for frequent procedure calls lowers overhead under load.
 - Streamlined report-generation code speeds up appointment confirmations.
 - Scalable optimized IR helps the platform serve more patients concurrently without a redesign.
-

3. Automated Toll Collection System

A transportation authority is implementing an automated toll collection system that identifies vehicles and processes payments electronically. The compiler embedded in the toll management software evaluates Boolean expressions and assignment statements for vehicle classification and transaction validation. During festival seasons, improper backpatching caused delays in payment verification and traffic management. The organization aims to redesign the compiler to improve transaction speed and operational efficiency.

Tasks:

- (i) Generate intermediate code for vehicle identification and toll payment processing. (K3)
- (ii) Explain the significance of backpatching in automated toll management systems. (K4)
- (iii) Analyze how compiler optimization improves large-scale transportation applications. (K5)

Answer:

(i) Intermediate Code:

```
t1 = vehicleClass
t2 = balance >= tollAmount[t1]
if t2 goto Lpay
goto Ldeny
Lpay: balance = balance - tollAmount[t1]
    call openBarrier(gateID) ; goto Lend
Ldeny: call flagVehicle(vehicleID)
Lend:
```

(ii) Explanation:

Significance of backpatching in automated toll management systems:

- The Boolean balance-sufficiency check generates jump instructions before the pay/deny blocks that are generated later in the code.
- Backpatching resolves these targets once the barrier-open and flag-vehicle blocks are emitted, keeping code generation a single forward pass.
- This ensures the correct branch executes immediately for each vehicle, addressing the delays in payment verification seen during festival traffic.
- It supports combining vehicle-classification lookups with the balance check without needing target addresses known in advance.

(iii) Analysis:

How compiler optimization improves large-scale transportation applications:

- Optimized code shortens the vehicle-identification-to-barrier-open path, directly reducing queue delays at toll gates.
- Eliminating redundant classification lookups per vehicle lowers per-transaction processing time.
- Efficient generated code supports higher vehicle throughput during peak/festival traffic without additional gates.
- It improves consistency of transaction validation across large-scale distributed toll infrastructure.

4. Intelligent Fire Detection and Alarm System

A safety engineering company is developing an intelligent fire detection system that continuously monitors smoke and temperature sensors in industrial buildings. The compiler processes Boolean expressions, assignment statements, and case statements to activate alarms and emergency protocols. During emergency situations, inefficient intermediate code execution delayed warning notifications and evacuation procedures. The software engineers plan to optimize the compiler for faster real-time response.

Tasks:

- (i) Develop intermediate code for fire detection and emergency alert logic. (K3)
- (ii) Explain how case statements efficiently handle multiple emergency conditions. (K4)
- (iii) Evaluate the role of compiler optimization in industrial safety systems. (K5)

Answer:

(i) Intermediate Code:

```
t1 = smokeLevel > smokeThreshold
t2 = temperature > tempThreshold
t3 = t1 and t2
if t3 goto Lfire
if t1 goto Lwarn
goto Lnormal
Lfire: alertLevel = "CRITICAL"
      call activateSprinklers(zoneID) ; call soundAlarm(zoneID) ; goto Lend
Lwarn: alertLevel = "WARNING" ; call soundAlarm(zoneID) ; goto Lend
Lnormal: alertLevel = "NORMAL"
Lend:
```

(ii) Explanation:

How case statements efficiently handle multiple emergency conditions:

- Distinct severity levels (normal/warning/critical) are dispatched as separate branches so each condition triggers only its appropriate response (alarm only vs. alarm plus sprinklers).
- This avoids running full emergency-response logic for lower-severity readings, keeping normal-condition checks lightweight.
- New emergency categories (e.g. a gas leak) can be added as additional branches.
- Response actions such as sprinkler activation and evacuation protocol stay isolated per severity case.

(iii) Analysis:

Role of compiler optimization in industrial safety systems:

- Optimized code shortens the sensor-to-alarm pipeline, addressing the delayed warning notifications seen during emergency situations.
- Removing redundant Boolean recomputation across the continuous monitoring loop reduces embedded processing load.

- Faster generated code is critical to trigger evacuation protocols within safety-mandated response times.
- It supports scaling detection across more zones/sensors in large industrial buildings without slowing the response.

5. AI-Based Customer Support Chatbot

A software company is building an AI-based customer support chatbot to handle user queries and provide automated responses. The compiler processes declarations, assignment statements, Boolean expressions, and procedure calls to analyze user requests and generate responses dynamically. During large-scale customer interactions, inefficient intermediate code generation caused delays in response delivery and reduced system performance. The organization plans to redesign the compiler to improve scalability and user experience.

Tasks:

- (i) Design intermediate code for chatbot query processing and response generation. (K3)
- (ii) Explain how procedure calls improve modularity in AI-based chatbot systems. (K4)
- (iii) Analyze the importance of compiler optimization in intelligent customer support applications. (K5)

Answer:

(i) Intermediate Code:

```

param userQuery
call classifyIntent, 1
t1 = intentCategory
if t1 == 'BILLING' goto L1
if t1 == 'TECHNICAL' goto L2
goto L3
L1: param userQuery ; call getBillingResponse, 1 ; goto Lend
L2: param userQuery ; call getTechnicalResponse, 1 ; goto Lend
L3: param userQuery ; call getGenericResponse, 1
Lend: call sendResponse(response)

```

(ii) Explanation:

How procedure calls improve modularity in AI-based chatbot systems:

- `classifyIntent()`, `getBillingResponse()` and `getTechnicalResponse()` are compiled as separate reusable procedures invoked per user query.
- The response-generation logic for each domain stays isolated, so updates to one intent's handling do not affect others.
- New intent-handling procedures can be added easily as the chatbot's scope grows.
- Reuse of common procedures such as `sendResponse()` avoids duplicated code across every intent branch.

(iii) Analysis:

Importance of compiler optimization in intelligent customer support applications:

- Optimized code lowers per-query processing latency, addressing the response delays seen during large-scale customer interactions.

- Efficient handling of frequent procedure calls (activation-record reuse/inlining) reduces overhead under high concurrent query volume.
- Streamlined intermediate code supports scaling the chatbot to more simultaneous users without proportional infrastructure growth.
- Faster generated code improves user experience by shortening the time to first response.

RUBRICS FOR CALCULATION

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations

Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
---------------------------	-----	---	---	-----------------------------------	-------------------

